

Overview of Conversational Web Services

<https://www.w3.org/TR/wsc10/>

A Web Service and the client application that invokes it may communicate multiple times to complete a single task. Also, multiple client applications might communicate with the same Web Service at the same time. Conversations provide a straightforward way to keep track of data between calls and to ensure that the Web Service always responds to the correct client.

Conversations meet two challenges inherent in persisting data across multiple communications:

1. Conversations uniquely identify a two-way communication between one client application and one Web Service so that messages are always returned to the correct client. For example, in a shopping cart application, a conversational Web Service keeps track of which shopping cart belongs to which customer. A conversational Web Service implements this by creating a unique conversation ID each time a new conversation is started with a client application.

2. Conversations maintain state between calls to the Web Service; that is, they keep track of the data associated with a particular client application between its calls to the service. Conversations ensure that the data associated with a particular client is saved until it is no longer needed or the operation is complete. For example, in a shopping cart application, a conversational Web Service remembers which items are in the shopping cart while the customer continues shopping. Maintaining state is also needed to handle failure of the computer hosting the Web Service in the middle of a conversation; all state-related data is persisted to disk so that when the computer comes up it can continue the conversation with the client application.

WebLogic Server manages this unique ID and state by creating a conversation context each time a client application initiates a new conversation. The Web Service then uses the context to correlate calls to and from the service and to persist its state-related data.

Conversations between a client application and a Web Service have three distinct phases:

Start—a client application initiates a conversation by involving the start operation of the conversation and web service. The web service in turn creates a context and an accompanying unique ID and starts the internal timer to measure the other time and the age of the conversation

Continue—After the client application has started the conversation, it invokes one or more continue operations to continue the conversation. The conversational Web Service uses the ID associated with the invoke to determine which client application it is conversing with, what state to persist, and which idle timer to reset. A typical continue operation would be one that requests more information from the client application, requests status, and so on.

Finish—a client application especially please invoke the finish operation when it has finished its conversation the web service then marks any data or resources associated with the conversation as deleted

Conversations typically occur between two weblogic: **Web Services** one is marked conversational and defines the start continue and finish operations and the other web service uses the **@ServiceClient** annotation to specify that it is a client of the conversational web service you can also invoke a conversational web service from standalone Java client although their descriptions are restrictions.

To enable effective communication between two business partners, they must agree on three important aspects.

1) Business Payload (What information is exchanged?)

Business payload refers to the actual business data exchanged between services.

Examples

- Customer order details
- Product information
- Invoice data
- Payment details
- Shipping information

Since web services commonly use XML, the business payload is usually defined using **XML Schemas (XSD)**.

2) Protocol (How information is exchanged?)

A protocol defines the rules for communication between services.

It specifies:

- Message format
- Header information
- Communication method
- Error handling mechanism
- Underlying transport protocol

3) Service Location (Where is the service available?)

To use a web service, the client must know:

- Service address (URL)
- Supported protocols
- Available operations

WSCL is an XML-based specification used to describe the sequence of interactions (conversation flow) between a client and a web service.

Business-Level Conversation

A business conversation defines:

1. Which XML documents are exchanged.
2. The sequence in which they are exchanged.

This sequence represents the **external visible behavior** of a web service.

Example: Online Shopping Process

**Customer Login —-> Search Product —-> Add product to Cart —->
Place Order —-> Payment —-> Order Confirmation**

Abstract Interface of a Web Service

The combination of Documents exchanged and Sequence of exchange defines the **Abstract Interface** of a web service.

An abstract interface describes: Supported operations, Message flow, Business interactions and hide Application logic, Database operations, Internal processing, Backend implementation

Example: Consider a Railway Reservation System.

Visible to User

1. Search Train
2. Check Availability
3. Book Ticket
4. Make Payment
5. Receive Ticket

Hidden Internally

- Database queries
- Seat allocation algorithms
- Payment verification
- Server-side processing

WSCL describes only the visible conversation flow.

Loose Coupling

Web services achieve flexibility by separating. company may change SOAP API to REST API, server location and Internal Database without changing the business conversation used by customers.

This independence is called **Loose Coupling**.

Benefits of Loose Coupling

- Easier system integration
- Vendor independence
- Service replacement without affecting clients
- Better scalability
- Dynamic service discovery

Web Services Conversation Language (WSCL).

Web Services Conversation Language (WSCL) is an XML-based language used to describe the sequence of interactions between web services and clients.

- Developed by Hewlett-Packard (HP) in 2001.
- Submitted to the W3C in 2002.
- Published as a W3C Note on March 14, 2002.
- Defines the **public behavior** of a web service.
- Specifies **what messages can be exchanged and in what order**.
- Does not describe internal implementation details or communication protocols.

Need for WSCL

WSDL describes:

- Operations offered by a web service.
- Input and output message formats.

However, WSDL does **not specify the sequence** in which messages must be exchanged. WSCL solves this problem by defining the **conversation flow** between service participants.

Objectives of WSCL

- Describe business-level conversations between web services.
- Define valid sequences of message exchanges.
- Support dynamic service discovery and integration.
- Promote loose coupling between interacting systems.
- Complement WSDL by adding conversation flow information.

Key Features of WSCL

- XML-based specification.
- Defines public processes of web services.
- Independent of implementation and transport protocols.
- Supports one-way and two-way interactions.
- Enables validation of message sequences.
- Supports integration with WSDL and XML Schema.

Elements of the WSCL Specification

There are four main elements to a WSCL specification:

- 1) Conversation**
- 2) Document Type Descriptions**
- 3) Interaction**
- 4) Transitions**

Document Type Descriptions define the structure and type of XML documents that a web service can **receive** and **send** during a conversation.

These descriptions specify:

- What kind of XML documents are accepted by the service.
- What kind of XML documents are returned as responses.
- The format and structure of business data exchanged between participants.

WSCL does **not store the actual XML schema definitions** inside the conversation specification.

Instead:

- XML Schemas (XSD files) are created separately.
- WSCL only references these schemas using their URL.
- This keeps conversation logic separate from data structure definitions.

The **interaction** between service-consumer and service-provider is achieved through XML document exchange.

A conversation definition language must be able to define all the input and output document types.

The document type definitions refer to the schema that the document corresponds to. They also serve to declare an id for the document that can be used within the rest of the conversation definition.

In a WSCL conversation definition, the document types are declared within the interaction definitions, either as an InboundXMLDocument or OutboundXMLDocument, depending on whether the document is expected as input or is produced as output in the interaction.

WSCL only references the schemas for the business payload of any messages.

InboundXMLDocument = Message received by the service.

OutboundXMLDocument = Message sent by the service.

A single interaction can contain **multiple OutboundXMLDocument** elements to represent different outcomes such as **Success**, **Failure**, **Error**, or **Out of Stock** responses.

For example, the following definition defines an input document that conforms to a purchase order schema defined in PurchaseOrderRQ.xsd.

```
<Interaction interactionType="ReceiveSend" id="RequestQuote">
```

```
  <InboundXMLDocument
```

```
    id="PurchaseOrderRQ"      request document ID
```

```
    hrefSchema="http://example.com/PurchaseOrder.xsd"/>
```

```
  <OutboundXMLDocument
```

```
    id="QuoteRS"              response document ID
```

```
    hrefSchema="http://example.com/Quote.xsd"/>
```

```
</Interaction>
```

Interactions model the actions of the conversation as document exchanges between conversation participants.

WSCL currently supports five types of interactions:

- **Send** (the service sends out an outbound document),
- **Receive** (the service receives an inbound document),
- **SendReceive** (the service sends out an outbound document, then expects to receive an inbound document in reply), and
- **ReceiveSend** (the service receives an inbound document and then sends out an outbound document).

The Empty interaction does not contain any documents exchanged, it is just used for modeling the start and end of a conversation.

An interaction is an exchange of one or two documents between a service and its client. WSCL only models business level interactions, i.e. it only specifies which business level documents are exchanged and does not model how this exchange is carried out by lower level messaging protocols.

The actual messaging may involve more than one message per business document exchange, e.g. an HTTP-POST as well as an HTTP-RESPONSE per business document. The messaging logic may even introduce additional transport level acknowledge messages, e.g. to achieve reliable messaging.

Receive and Send (one-way interactions), ReceiveSend and SendReceive (two-way interactions), Empty (see section on initial and final interactions).

One-way Interactions:

These interactions represent a single one-way message being sent or received by a participant.

There are two subtypes of one-way interactions: Receive and Send.

The Send interaction represents a document sent out by a participant, and on the other hand a Receive interaction represents a document being received by a participant.

The following example represents a Receive interaction that receives a login document:

```
<Interaction interactionType="Receive" id="LoginRegInput">  
  <InboundXMLDocument id="LoginRequestData"  
    hrefSchema=http://foo.org/LoginRequestData.xsd />  
</Interaction>
```

A Receive interaction must contain one InboundXMLDocument element. A Send interaction must contain one OutboundXMLDocument.

Two-way Interactions:

Two-way interactions can have one of two forms, **SendReceive** or **ReceiveSend**, depending on whether the participant sends out a message for which it gets a response (SendReceive) or whether the participant responds to a request that it receives (ReceiveSend).

ReceiveSend Interactions: Each such interaction is the logical unit of receiving a request and then returning a response. The interaction is not complete until the response has been sent. `<Interaction interactionType="ReceiveSend" id="Quotation">`

```
<InboundXMLDocument id="PurchaseOrderRQ"
  hrefSchema="http://foo.org/PurchaseOrderRQ.xsd">
```

```
</InboundXMLDocument>
```

```
<OutboundXMLDocument id="InvoiceRS"
  hrefSchema="http://foo.org/InvoiceRS.xsd">
```

```
</OutboundXMLDocument>
```

```
</Interaction>
```

A ReceiveSend interaction can specify more than one OutboundXMLDocument.

This allows the modeling of cases where there are multiple types of response messages that a service might return in response to a specific request. Having additional possible responses is mainly used for error messages.

If a ReceiveSend interaction specifies more than one outbound document type, at run-time only one of them is being exchanged. For modeling the cases where a request message initiates several response messages, several interactions have to be used.

SendReceive Interaction: Each such interaction is the logical unit of sending a request and then receiving a response. The interaction is not complete until the response has been exchanged. A SendReceive interaction can specify more than one InboundXMLDocument.

```
<Interaction interactionType="SendReceive" id="Payment">
  <OutboundXMLDocument id="Invoice"
    hrefSchema="http://foo.org/InvoiceRS.xsd">
  </OutboundXMLDocument>
  <InboundXMLDocument id="Payment"
    hrefSchema="http://foo.org/Payment.xsd">
  </InboundXMLDocument>
</Interaction>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<Conversation name="OrderProcessingConversation">

  <!-- SEND -->
  <Interaction id="RegisterInterest" interactionType="Send">
    <OutboundXMLDocument href="InterestNotification.xsd"/>
  </Interaction>

  <!-- SENDRECEIVE -->
  <Interaction id="Login" interactionType="SendReceive">
    <OutboundXMLDocument href="LoginRequest.xsd"/>
    <InboundXMLDocument href="LoginResponse.xsd"/>
  </Interaction>

  <!-- SENDRECEIVE -->
  <Interaction id="PlaceOrder" interactionType="SendReceive">
    <OutboundXMLDocument href="OrderRequest.xsd"/>
    <InboundXMLDocument href="OrderConfirmation.xsd"/>
  </Interaction>

  <!-- RECEIVE -->
  <Interaction id="ReceiveInvoice" interactionType="Receive">
    <InboundXMLDocument href="InvoiceNotification.xsd"/>
  </Interaction>
```

Initial and Final Interactions

Part of defining the possible ordering of interactions is the specification of the first and last interactions of a conversation.

In WSCL, this is done by the attributes **initialInteraction** and **finalInteraction** of the Conversation element.

The attribute initialInteraction references the ID of the first interaction to be executed in the conversation.

The attribute finalInteraction references the ID of the last interaction to be executed.

```
<Conversation name="ExampleConversation"  
  initialInteraction = "Login"  
  finalInteraction = "Purchase" >
```

Transitions specify the ordering relationships between interactions. A transition specifies a source interaction, a destination interaction, and, optionally, a document type of the source interaction as an additional condition for the transition.

A conversation can proceed from one interaction to another as allowed by the permissible sequencing defined in the transition elements.

```
<Transition>
```

```
  <SourceInteraction href="Invoice"/>
```

```
  <DestinationInteraction href="Receipt"/>
```

```
  <SourceInteractionCondition href="InvoiceRS"/>
```

```
</Transition>
```

SourceInteraction references an interaction that can precede the **DestinationInteraction** when the conversation is executed.

Similarly, **DestinationInteraction** references one of the interactions that can follow the **SourceInteraction** when the conversation is executed. Together, all transitions specify all possible sequences of the interactions.

SourceInteractionCondition is an additional constraint on the transition. It is needed when the **SourceInteraction** specifies more than one possible document to be exchanged and the type of document exchanged has an influence on the possible next interactions.

SourceInteractionCondition references an **OutboundXMLDocument** of the **SourceInteraction** when it is a **ReceiveSend** interaction, or an **InboundXMLDocument** from the **SourceInteraction** when it is a **SendReceive** interaction.

If no **SourceInteractionCondition** is listed, **DestinationInteraction** can be triggered independent of the types of documents exchanged in **SourceInteraction**.

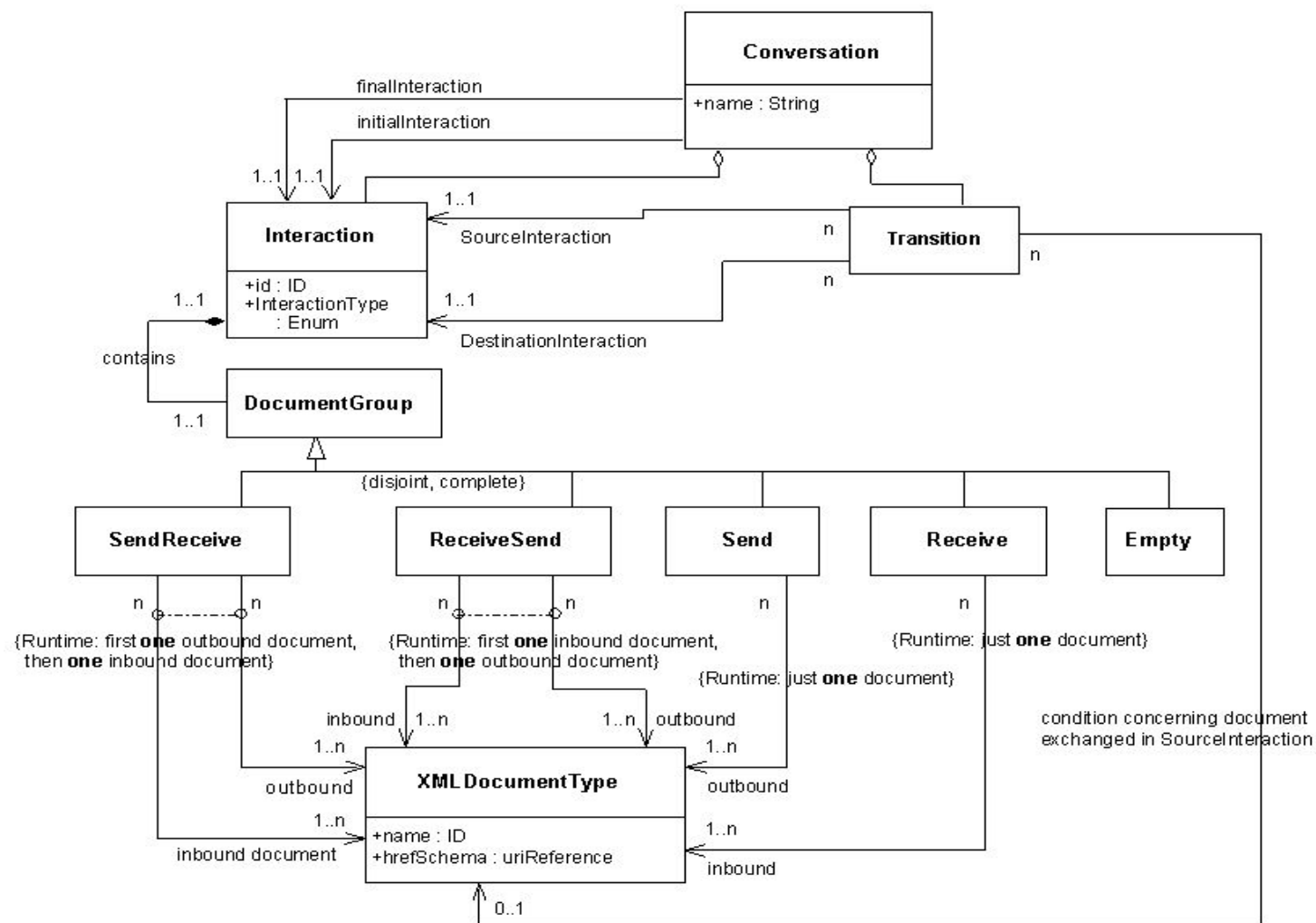
There is one important limitation to specifying transitions without SourceInteractionCondition: If there exists a transition from SourceInteraction A to DestinationInteraction B that specifies a SourceInteractionCondition, then it is not possible to also specify a transition from SourceInteraction A to DestinationInteraction B without a SourceInteractionCondition.

Transitions define all the permissible orders of interactions but do not specify under which condition which permissible sequence is chosen.

This is determined by the internal application logic of both participants, often based on back-end information; for example, in the example it is the buyer who decides whether the CatalogInquiry interaction is followed by a Quote interaction or a Purchase interaction, by sending either a PurchaseRQ or QuoteRQ document. A PurchaseRQ document triggers the Purchase interaction; a QuoteRQ document triggers the Quote interaction.

The transitions defined in a conversation can be represented by a directed graph or a UML activity diagram. A UML activity diagram represents interactions by action states.

These action states model the inbound and outbound document exchanges as actions. The first of these document exchanges also triggers the transition to the action state itself and, therefore, could be drawn as an event on the transition. SourceInteractionConditions appear as additional conditions on the transitions.



Each conversation definition has the root element ***Conversation***.

The Conversation element contains the following two sub-elements:

- ***ConversationInteractions*** lists all Interaction elements.
- ***ConversationTransitions*** lists all Transition elements.

The Conversation element contains the following attributes:

- ***initialInteraction*** and ***finalInteraction*** reference the initial and final interactions of the conversation.
- ***name*** is the name of a conversation. The name is the shared piece of information needed by both parties so they realize the same conversation type in their service implementation. This name would also appear in the header elements of the actual messages exchanged. WSCL does not specify how to name conversations. Conversation names do not have to be URIs. If a conversation is published in a UDDI directory, the conversation name could be a tModel key.
- ***version (optional)*** is the version of the conversation. If no version number is given, the name of the conversation must be unique.
- ***targetNamespace (optional)*** is the namespace of this conversation as it should be used when elements of this conversation are referenced in other XML documents.
- ***hrefSchema (optional)*** is the URL of the file containing this conversation definition.
- ***description (optional)*** is the textual description of the conversation.

<?xml version="1.0" encoding="UTF-8"?>

<Conversation name="StoreFrontServiceConversation" version="1.01"
xmlns="http://www.w3.org/2002/02/wscl10"
initialInteraction="Start" finalInteraction="End"
targetNamespace="http://example.com/conversations/StoreFront101"
hrefSchema="http://example.com/schema_files/StoreFront101.wscl"
description="Conversation for a Store Front Service" >

<ConversationInteractions>

list of all the interactions

</ConversationInteractions>

<ConversationTransitions>

list of all the transitions

</ConversationTransitions>

</Conversation>

<conversation>

<interaction id="Login"/>

<interaction id="BrowseProduct"/>

<interaction id="AddToCart"/>

<interaction id="Payment"/>

<interaction id="OrderConfirm"/>

<transition fromInteraction="Login" toInteraction="BrowseProduct"/>

<transition fromInteraction="BrowseProduct" toInteraction="AddToCart"/>

<transition fromInteraction="AddToCart" toInteraction="Payment"/>

<transition fromInteraction="Payment" toInteraction="OrderConfirm"/>

</conversation>

Mapping to Protocol Bindings and Error Handling

When executing a conversation, we can expect two levels of errors: those concerning the document content and those concerning the specified conversations.

Errors of the first type occur if a participant receives the correct type of document with the correct document structure but invalid values; for example, invalid username and password. Such errors need to be handled by the business logic, and the conversation definition and document schemas have to allow their communication for example, a document type for invalid login, or invalid login fields in a general login response document type.

Errors concerning the document types used, the order of interactions, or the conversations to be used need to be handled by the communication protocol.

The protocol might have predefined error messages that signal "unexpected document type" or "unknown conversation." The protocol also has to define whether a conversation instance continues or terminates after the exchange of an invalid document type, or what happens if one of the participants does not react for a long time. WSCL does not specify how interactions are bound to a specific communication protocol because protocol bindings are outside the scope of WSCL. **There are two different approaches for providing binding information.**

- The protocol binding is determined by a specific Web services framework used and is applicable to all services and all messages exchanged.
- A special protocol binding description language is used to describe the protocol bindings for the individual interactions or conversations. An example of such a protocol binding description language is the protocol binding part of WSDL

Combining WSCL and WSDL to Describe Web Services

		WSDL	WSCL
Abstract Interfaces	choreography	<i>out of scope</i>	Transition
	messages	Operation	Interaction
Protocol Bindings		Binding	<i>out of scope</i>
Concrete Services		Service	<i>out of scope</i>

WSDL	WSCL
Port Type	Conversation
Operation: - One-way - Request-response - Solicit-response - Notification	Interaction*: - Receive - ReceiveSend - SendReceive - Send
Input	InboundXMLDocument
Output, Fault	OutboundXMLDocument
Names of Operation, Input, Output, Fault	ID of Interaction, InboundXMLDocument, OutboundXMLDocument
Message	URL of XML schema (WSCL delegates the specification of the payload entirely to an external XML schema, whereas WSDL directly uses XML data types)

There are three approaches for combining WSDL and WSCL descriptions of a Web service:

Adding protocol bindings in WSDL to a conversation described in WSCL: If the abstract interface is already completely described by a WSCL document, we can use the WSDL Binding elements to describe the protocol binding; for example:

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions name="MyStoreFrontService"
  xmlns="http://schemas.xmlsoap.org/wsdl/"
  targetNamespace=...
  xmlns:conv="http://example.com/Conversations/StoreFront.cdl" >
  <binding name="StoreFrontServiceConversationBinding"
    type="conv:StoreFrontServiceConversation">
    <soap:binding style="document"/>
    <operation name="conv:Login">
      <soap:operation soapAction="Login">
    </operation>
    <operation name="conv:Registration">
      <soap:operation soapAction="Registration">
    </operation>
    ... for all other interactions
  </binding>

  <service name="MyStoreFrontService">
    <port name="MyStoreFrontServiceAccessPoint"
      binding="StoreFrontServiceConversationBinding">
      <soap:Address location="http://mystore.com/storefront" />
    </port>
  </service>
</definitions>
```

Adding choreography to a WSDL port type description: If a port type is already described by a WSDL document, we can describe the choreography in an additional WSCL document that only contains transition elements. The sub-elements SourceInteraction and DestinationInteraction refer to the names of operations from the WSDL document. The sub-element SourceInteractionCondition refers to an output or fault message of the operation. WSCL uses attributes of type HREF to refer to other elements. Therefore, to refer to operations from a WSDL document, the WSCL schema needs to be slightly adapted to also accept values of type QNAME.

Providing a full WSDL and WSCL description for the same port type/conversation: We can also express which documents get exchanged by both a WSDL port type and a WSCL conversation with the conversation also describing the choreography used.

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions name="MyStoreFrontService"
  xmlns="http://schemas.xmlsoap.org/wsdl/"
  targetNamespace=...
  xmlns:xsd1="http://example.com/types/LoginRQ.xsd"
  xmlns:xsd2="http://example.com/types/ValidLoginRS.xsd"
  xmlns:xsd3="http://example.com/types/InvalidLoginRS.xsd" >
```

```
  <message name="LoginRequestDocument">
    <part name="body" element="xsd1:LoginRQElement" />
  </message>
  <message name="ValidLoginDocument">
    <part name="body" element="xsd2:ValidLoginRSElement" />
  </message>
  <message name="InvalidLoginDocument">
    <part name="body" element="xsd3:InvalidLoginRSElement" />
  </message>
```

description together. It only provides the **WSDL abstract interface (portType)** for the conversation.)

```
  <portType name="StoreFrontServiceConversation" >
    <operation name="Login">
      <input name="LoginRQ" message="LoginRequestDocument"/>
      <output name="ValidLoginRQ" message="ValidLoginDocument"/>
      <fault name="InvalidLoginRQ" message="InvalidLoginDocument"/>
    </operation>
  </portType>
</definitions>
```

Business Process Management (BPM)

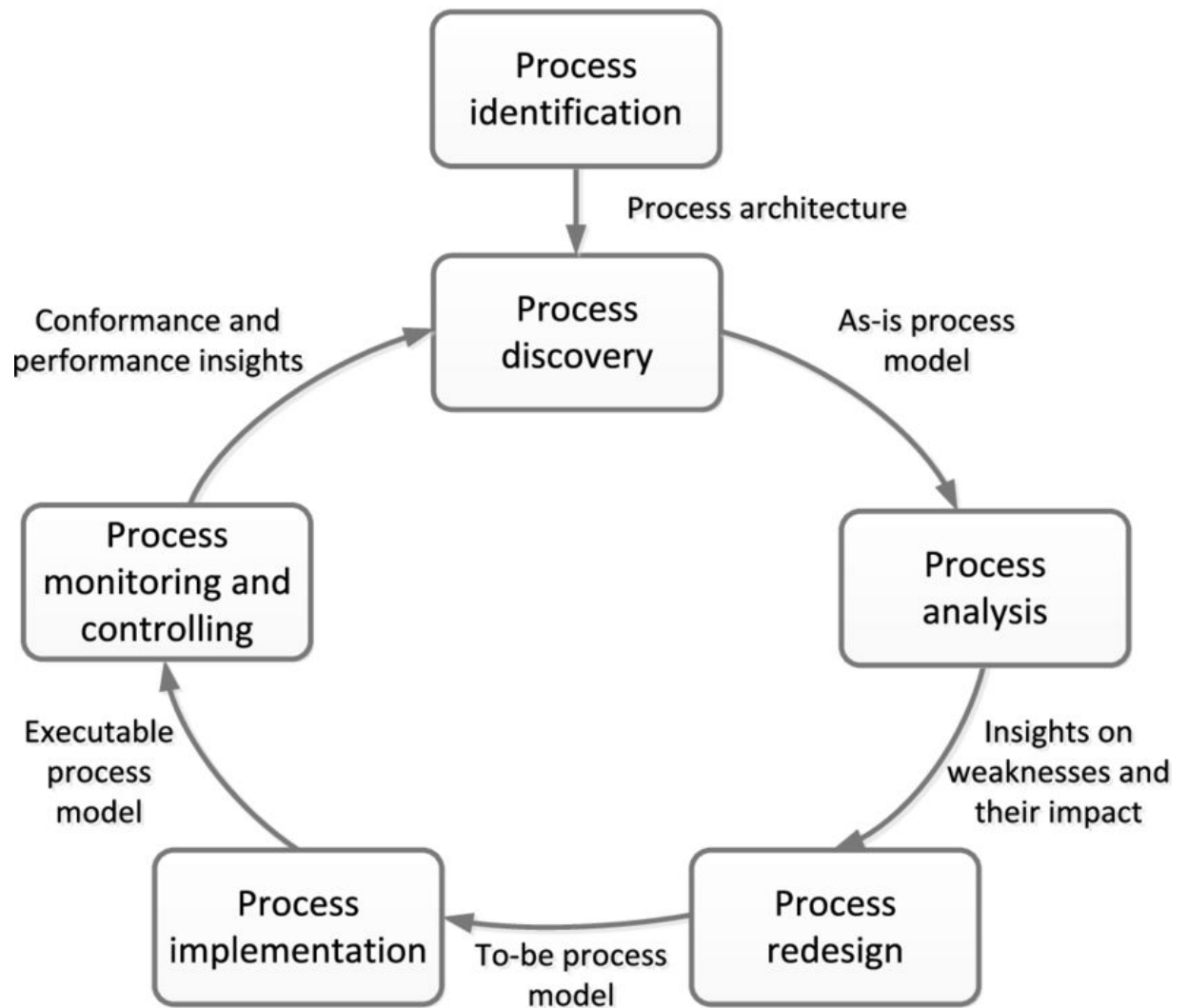
Business Process Management (BPM) is a systematic approach used to discover, design, model, execute, monitor, analyze, improve, and optimize business processes to achieve organizational goals efficiently. BPM focuses on managing and improving end-to-end business processes rather than individual tasks.

Definition- BPM employs methods to discover, model, analyze, measure, improve, and optimize business strategy and processes.

Why BPM is Important

- Improve operational efficiency
- Reduce costs
- Increase productivity
- Enhance customer satisfaction
- Improve transparency
- Support digital transformation
- Automate repetitive activities
- Standardize business operations

BPM Lifecycle



Stage 1: Process Design

Activities:

- Identify business process
- Define workflow steps
- Define task owners
- Define business rules

Example:

**Customer Order Process —-> Receive Order—->
Verify Payment—-> Ship Product**

Stage 2: Process Modeling

Visual representation of business processes.

Common tools:

- Flowcharts
- BPMN Diagrams
- Workflow diagrams

Purpose:

- Understand process flow
- Identify bottlenecks
- Improve communication

Stage 3: Process Execution

Implementation of designed process.

Methods:

- Manual execution
- Automated execution
- Hybrid execution

Example:

**Online Purchase—-> Automatic Payment Verification—->
Automatic Invoice Generation**

Stage 4: Process Monitoring

Track process performance using KPIs.

Metrics:

- Processing time
- Error rate
- Customer satisfaction
- Cost per transaction

Stage 5: Process Optimization

Continuous improvement based on monitoring results.

Activities:

- Remove bottlenecks
- Automate repetitive tasks
- Improve workflow efficiency

Types of BPM

IBM identifies three major BPM categories.

A. Integration-Centric BPM

Focuses on system-to-system integration with minimal human involvement.

Examples

- ERP Integration
- CRM Integration
- HR Management Systems

Uses:

- APIs
- Databases
- Enterprise Applications

B. Human-Centric BPM

Focuses on workflows requiring human approval and decision making.

Examples

- Leave Approval
- Expense Approval
- Recruitment Process

Workflow

Employee—> Manager Approval—> HR Approval

C. Document-Centric BPM

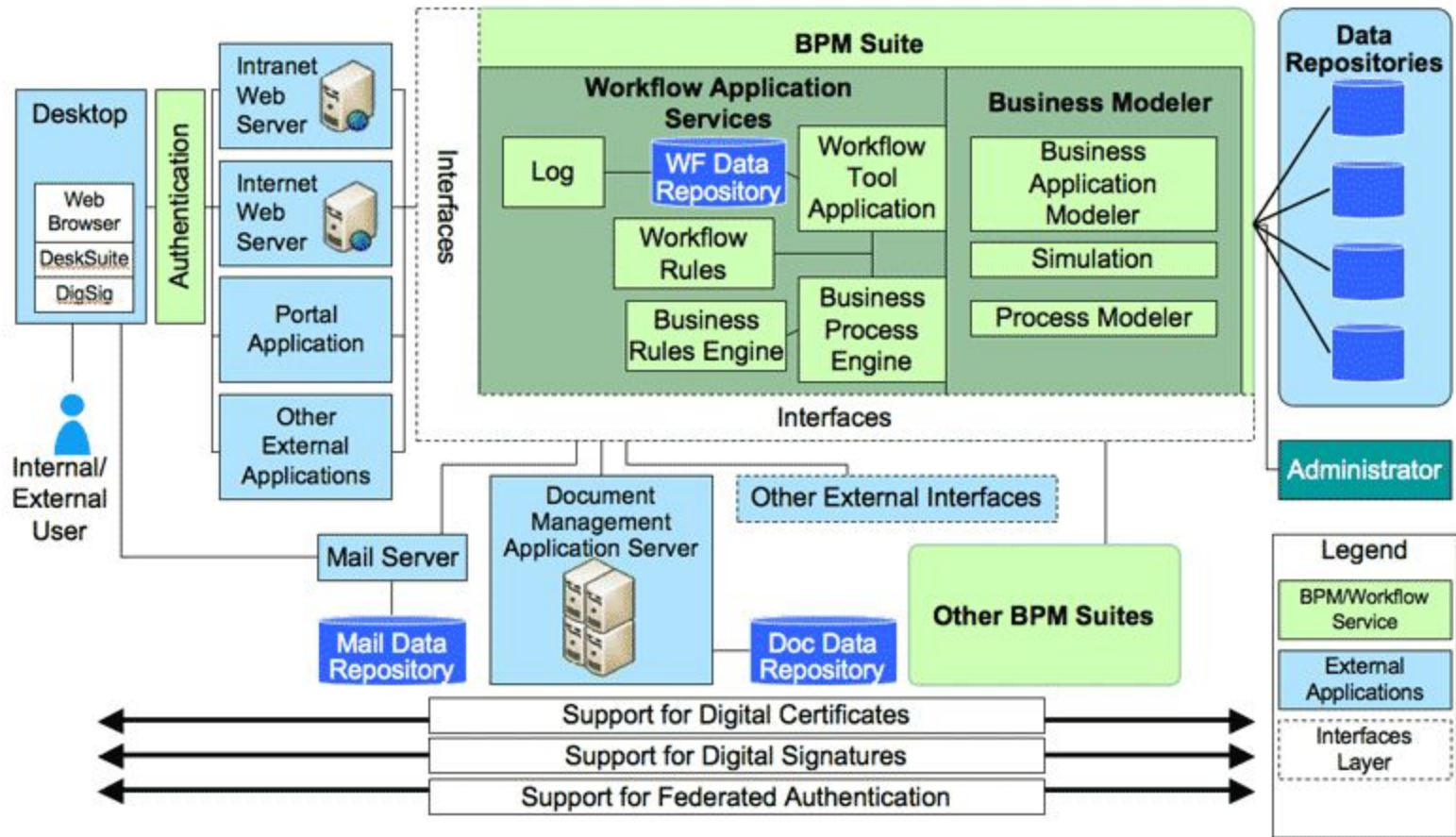
Focuses on documents moving through various approval stages.

Examples

- Contracts
- Purchase Orders
- Legal Documents

Workflow

Document Creation—> Review—> Approval—> Storage



Business Process Modeling Notation (BPMN)

The Business Process Modeling Notation (BPMN) is visual modeling language for business analysis applications and specifying enterprise process workflows, which is an open standard notation for graphical flowcharts that is used to define business process workflows.

BPMN is derived from the synthesis of multiple business modeling notations. Originally published by the Business Process Management Initiative (BPMI) in 2004, BPMN is now being maintained by the OMG since the two organizations merged in 2005. BPMI merged with OMG, the Object Management Group. A BPMN Specification document was released by OMG in February, 2006. Version 2.0 of BPMN was developed in 2010, and the actual version of the specification was released in December 2013.

BPMN allows us to capture and document business processes of an organization in a clear and consistent way that ensures relevant stakeholders, such as, process owners and business users are involved in the process. Thus, the team can response to any issues identified in the processes more effectively. BPMN provide comprehensive and yet rich notations that can easily be understood by both technical and non-technical stakeholders.

The Goal of BPMN

- Technical experts responsible for process implementation
- Business analysts who create and improve the processes
- Managers who monitor and control the processes

BPMN Notation

<https://edrawmax.wondershare.com/symbols/bpmn-symbols.html>

There are five basic categories of BPMN elements. Each of them represent a unique aspect of business process.

Swimlanes

Swimlanes are graphical containers that represent participants of a process. There are two types of swimlanes - pools and lanes.

Pools represent participants in a business process. It can be a specific entity (e.g. department) or a role (e.g. assistant manager, doctor, student, vendor).

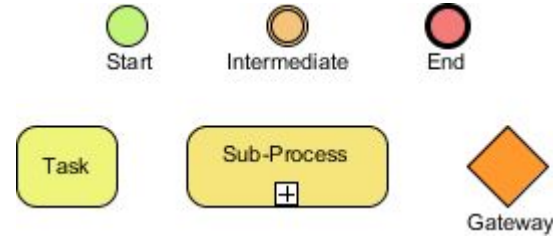
Inside a pool, there are flow elements. They represent the works that the pool needs to perform under the process being modeled. However, there is one kind of pool that has no content at all. It is known as the blackbox pool. Blackbox pool is often used when modeling entities external to the business process. As it is external, its internal flow does not have any impact on the process being modeled

Lanes are sub-partition of pools. For instance, when you have a pool Department, you may have Department Head and General Clerk as lanes. Same as pools, you can use lanes to represent specific entities or roles who are involved in the process.

Lanes may contain other lanes to form a nested structure when needed.

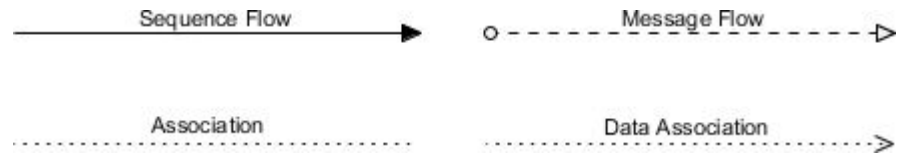
Flow Elements

Flow elements are elements that connect with each other to form business workflows. Flow elements are the primary elements that define the behavior of a process. There are three kinds of flow elements: Events, Activities and Gateways.



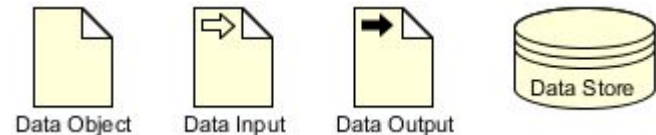
Connecting Objects

Flow objects are not isolated, but rather connected in order to form a flow. The connectors that connect the flow objects are called connecting objects. There are four kinds of connecting objects: Sequence flows, message flows, associations and data associations.



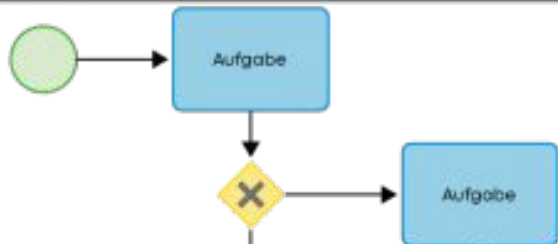
Data

Data is mainly information needed or produced when executing a business process. There are four kinds of data: Data objects, data inputs, data outputs and data stores.



Pool

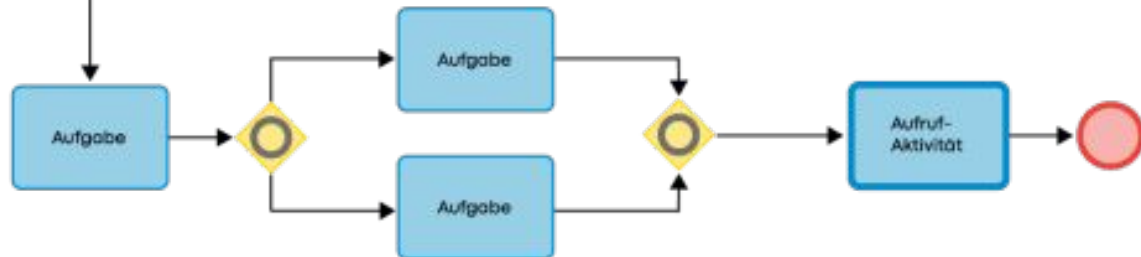
Lane 1



Lane 2



Lane 3



Workflow Management Systems (WFMS)

A workflow can be seen as a model for a large number of work processes that are repeated over and over. It is a flow chart for work processes. A workflow essentially handles the following aspects:

- Defining the **trigger** for the upcoming tasks
- Summarizing the **sequence** of tasks into a **meaningful workflow**
- **Determining** the **departments** and **persons** involved in the process
- **Identifying the relationships to and dependencies on other work processes in different departments.**

Workflows organize and structure all recurring work processes of a company.

Who approves what, when, where, how and under which conditions – this is what the workflow describes. Even exceptions and errors, such as malfunctions and people not being available, are identified and regulated by workflows. The focus is always on efficiency, simplification and achieving goals.

These automated helpers are an integral part of automation in many business software applications. For example, workflows can be found in the following areas:

- Incoming invoice management (e.g. for approval processes),
- Contract management (e.g. for signature processes),
- HR work (e.g. for the onboarding process) or
- Contract lifecycle management.

Workflows structure and organize all of these work processes, ensuring that the defined steps are followed and controlling the flow of documents as well as the overall coordination.

Ideally, a digital workflow management system (WfMS) controls all of the workflows in the company. In other words, it's also about task management. A system of this kind comprises at least three key features and functions:

Modeling and simulation

One of the central workflow management functions concerns the need for a graphical representation of workflows consisting of many work steps for the sake of easier handling.

Instantiation and execution

The workflow management system must instantiate precise and case-specific processes so that it will later be able to execute the workflow models. In this context, instantiate means that the workflow management system has an “idea” of which data, people, applications (programs), etc. it can interact with via the workflow model.

Monitoring and analysis

The key features and functions of workflow management systems also include the monitoring of current work steps and the analysis of processes that have already been carried out. This involves answering questions using the data generated in the workflows.

BPEL (Business Process Execution Language)

<https://www.oracle.com/technical-resources/articles/matjaz-bpel.html>

<https://www.oracle.com/technical-resources/articles/matjaz-bpel-advanced.htm>

!

Business Process Execution Language for Web Services (BPEL or BPEL4WS) is a language used for the definition and execution of business processes using Web services.

BPEL enables the top-down realization of Service Oriented Architecture (SOA) through composition, orchestration, and coordination of Web services.

BPEL provides a relatively easy and straightforward way to compose several Web services into new composite services called ***business processes***.

BPEL builds on the foundation of XML and Web services; it uses an XML-based language that supports the Web services technology stack, including SOAP, WSDL, UDDI, WS-Reliable Messaging, WS-Addressing, WS-Coordination, and WS-Transaction.

BPEL represents a convergence of two early workflow languages; Web Services Flow Language (WSFL) and XLANG.

WSFL was designed by IBM and is based on the concept of directed graphs. XLANG, a block-structured language, was designed by Microsoft. BPEL combines both approaches and provides a rich vocabulary for description of business processes.

The first version of BPEL was developed in August 2002.

Since then, many major vendors have joined (including Oracle), resulting in several modifications and improvements, and the adoption of version 1.1 in March 2003.

In April 2003, BPEL was submitted to the Organization for the Advancement of Structured Information Standards (OASIS) for standardization purposes, and the Web Services Business Process Execution Language Technical Committee (WSBPEL TC) was formed.

BPEL is used to standardize enterprise application integration as well as to extend the integration to previously isolated systems.

Web services usually expose operations of certain applications or information systems. Consequently, combining several Web services actually involves the integration of the underlying applications and their functionalities.

Web services can be combined in two ways:

- Orchestration
- Choreography

In **orchestration**, which is usually used in private business processes, a central process (which can be another Web service) takes control of the involved Web services and coordinates the execution of different operations on the Web services involved in the operation. The involved Web services do not "know" (and do not need to know) that they are involved in a composition process and that they are taking part in a higher-level business process. Only the central coordinator of the orchestration is aware of this goal, so the orchestration is centralized with explicit definitions of operations and the order of invocation of Web services.

Choreography, in contrast, does not rely on a central coordinator. Rather, each Web service involved in the choreography knows exactly when to execute its operations and with whom to interact. Choreography is a collaborative effort focusing on the exchange of messages in public business processes. All participants in the choreography need to be aware of the business process, operations to execute, messages to exchange, and the timing of message exchanges.

From the perspective of composing Web services to execute business processes, orchestration is a more flexible paradigm and has the following advantages over choreography:

- The coordination of component processes is centrally managed by a known coordinator.
- Web services can be incorporated without their being aware that they are taking part in a larger business process.
- Alternative scenarios can be put in place in case faults occur.

BPEL supports two different ways of describing business processes that support orchestration and choreography:

- **Executable processes** allow you to specify the exact details of business processes. They follow the orchestration paradigm and can be executed by an orchestration engine.
- **Abstract business protocols** allow specification of the public message exchange between parties only. They do not include the internal details of process flows and are not executable. They follow the choreography paradigm.

A BPEL process consists of steps; each step is called an "activity." BPEL supports primitive as well as structure activities. *Primitive activities represent basic constructs and are used for common tasks*, such as the following:

- Invoking other Web services, using `<invoke>`
- Waiting for the client to invoke the business process by sending a message, using `<receive>` (receiving a request)
- Generating a response for synchronous operations, using `<reply>`
- Manipulating data variables, using `<assign>`
- Indicating faults and exceptions, using `<throw>`
- Waiting for some time, using `<wait>`
- Terminating the entire process, using `<terminate>`

We can then combine these and other primitive activities to define complex algorithms that specify exactly the steps of business processes. To combine primitive activities, BPEL supports several **structure activities**. The most important are

- Sequence(`<sequence>`), which allows us definition of a set of activities that will be invoked in an ordered sequence
- Flow(`<flow>`) for defining a set of activities that will be invoked in parallel
- Case-switch construct(`<switch>`) for implementing branches
- While (`<while>`) for defining loops
- The ability to select one of several alternative paths, using `<pick>`

Each BPEL process will also define partner links, using `<partnerLink>`, and declare variables, using `<variable>`.

In develop the sample BPEL process, you will go through the following steps:

- Get familiar with the involved Web services

- Define the WSDL for the BPEL process

- Define partner link types

- Develop the BPEL process:

 - Define partner links

 - Declare variables

 - Write the process logic definition.

To develop a BPEL process corresponding to the activity diagram, you first have to define the process Web Services Definition Language (WSDL), including the port types, operations, and messages.

BPEL business processes need to handle faults appropriately and may also need to signal faults themselves.

Faults in BPEL can be from various sources:

- A BPEL process can explicitly signal (throw) a fault.
- A fault can occur when the BPEL process invokes a Web service operation. The operation might return a WSDL fault message, which results in a BPEL fault.
- A fault can be thrown automatically by the BPEL runtime environment, either due to a certain condition in the BPEL process itself (such as a join failure), as a consequence of error conditions in the runtime environment, or related to network communication or other reasons. For such situations, BPEL defines several standard faults.

Signaling Faults

A business process sometimes needs to signal a fault explicitly. Therefore, BPEL provides the `<throw>` activity

The business process can handle the fault through one or more fault handlers. Within a fault handler, the business process defines custom activities that should recover from the fault and possibly reverse the partial (unsuccessful) work of the activity where the fault has occurred.

Within the fault handlers, you specify several `<catch>` activities where you indicate which fault you want to catch and handle. Within a fault handler, you must specify at least one `<catch>` or `<catchAll>`. The `<catchAll>` activity can be specified only once within a fault handler.

Usually you specify several `<catch>` activities where you will handle specific faults and use `<catchAll>` to handle all other faults. To specify which fault you would like to handle, you must specify at least one of the following:

- faultName**, which specifies the name of the fault to handle

- faultVariable**, which specifies the variable type used for fault data

In complex BPEL processes, many faults can occur. Handling them in a single `<faultHandlers>` section can become quite complex. It would be useful if you could specify more than one `<faultHandlers>` section in a BPEL process. Particularly in complex processes, it would be great if you could specify different fault handler sections for different parts of the process. This is possible with **scopes**.

Fault Sources

Explicit Fault - Raised by process itself.

Service Fault - Returned by web service.

Runtime Fault - Generated by BPEL engine.

A Scope divides a large process into smaller logical sections.

Scope Contains

- Variables
- Fault Handlers
- Event Handlers
- Compensation Handlers
- Correlation Sets

Example :

```
<scope>  
  <variables/>  
  <faultHandlers/>  
  <eventHandlers/>  
  <compensationHandler/>  
</scope>
```

BPEL Deployment

A BPEL process is deployed to a BPEL engine such as:

- Oracle BPEL Process Manager
- Oracle SOA Suite
- Apache ODE

Deployment requires:

- BPEL File
- WSDL File
- Process Descriptor XML

Student Result Processing using BPEL

To create a simple BPEL process that receives a Student ID, invokes a Marks Service and a Grade Service, and returns the result.

Inside the bpel folder create:

MarksService.wsdl

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions
  xmlns="http://schemas.xmlsoap.org/wsdl/"
  xmlns:tns="http://example.com/marks"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  targetNamespace="http://example.com/marks">
```

```
  <message name="MarksRequest">
    <part name="studentId" type="xsd:string"/>
  </message>
  <message name="MarksResponse">
    <part name="marks" type="xsd:int"/>
  </message>
```

```
  <portType name="MarksPortType">
    <operation name="getMarks">
      <input message="tns:MarksRequest"/>
      <output
message="tns:MarksResponse"/>
    </operation>
  </portType>
</definitions>
```

Create GradeService.wsdl

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<definitions
```

```
  xmlns="http://schemas.xmlsoap.org/wsdl/"
```

```
  xmlns:tns="http://example.com/grade"
```

```
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
```

```
  targetNamespace="http://example.com/grade">
```

```
<message name="GradeRequest">  
  <part name="marks" type="xsd:int"/>  
</message>
```

```
<message name="GradeResponse">  
  <part name="grade" type="xsd:string"/>  
</message>
```

```
<portType name="GradePortType">  
  <operation name="calculateGrade">  
    <input message="tns:GradeRequest"/>  
    <output  
      message="tns:GradeResponse"/>  
    </operation>  
  </portType>  
</definitions>
```

Create Main BPEL File

StudentResultProcess.bpel

```
<?xml version="1.0"  
encoding="UTF-8"?>
```

```
<process  
  name="StudentResultProcess"
```

```
  targetNamespace="http://example.co  
m/studentresult"
```

```
  xmlns="http://docs.oasis-open.org/ws  
bpel/2.0/process/executable">
```

```
  <sequence>
```

```
    <receive  
      name="ReceiveStudentID"  
      createInstance="yes"/>
```

```
    <invoke  
      name="GetMarks"  
      partnerLink="MarksService"  
      operation="getMarks"/>
```

```
    <invoke  
      name="GetGrade"  
      partnerLink="GradeService"  
      operation="calculateGrade"/>
```

```
    <reply  
      name="SendResult"/>
```

```
  </sequence>
```

```
</process>
```

The screenshot shows the Eclipse IDE interface with the following components:

- Project Explorer:** Displays the project structure. The selected file is `StudentResultProcess.bpel` located at `src/main/webapp/bpel`.
- Editor:** Displays the BPEL XML code for `StudentResultProcess.bpel`. The code is as follows:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <process
4   name="StudentResultProcess"
5   targetNamespace="http://example.com/studentresult"
6   xmlns="http://docs.oasis-open.org/wsbpel/2.0/process/executable">
7
8   <sequence>
9
10    <receive
11      name="ReceiveStudentID"
12      createInstance="yes"/>
13
14    <invoke
15      name="GetMarks"
16      partnerLink="MarksService"
17      operation="getMarks"/>
18
19    <invoke
20      name="GetGrade"
21      partnerLink="GradeService"
22      operation="calculateGrade"/>
23
24    <reply
25      name="SendResult"/>
26
27  </sequence>
28
29 </process>
```
- Outline:** Displays a message: "There is no active editor that provides an outline."
- Bottom Panel:** Contains the **Problems**, **Servers**, **Terminal**, **Data Source Explorer**, **Properties**, and **Console** views. The **Servers** view shows the message: "No servers are available. Click this link to create a new server..."
- Status Bar:** Shows the text "Writeable", "Insert", and "12 : 22 : 284".
- System Tray:** Shows the Windows taskbar with the search bar, taskbar icons, and system clock (4:10 PM, 6/20/2026).

	apache-ode-war-1.3.4.zip.md5	2010-06-09 14:25	76
	apache-ode-war-1.3.4.zip.sha1	2010-06-09 14:25	78
	apache-ode-war-1.3.5.zip	2014-10-30 00:31	35M
	apache-ode-war-1.3.5.zip.asc	2014-10-30 00:31	851
	apache-ode-war-1.3.5.zip.md5	2014-10-30 00:31	32
	apache-ode-war-1.3.5.zip.sha1	2014-10-30 00:31	40
	apache-ode-war-1.3.6.zip	2014-10-30 00:31	42M
	apache-ode-war-1.3.6.zip.asc	2014-10-30 00:31	851
	apache-ode-war-1.3.6.zip.md5	2014-10-30 00:31	32
	apache-ode-war-1.3.6.zip.sha1	2014-10-30 00:31	40
	apache-ode-war-1.3.7.zip	2017-07-06 07:55	44M
	apache-ode-war-1.3.7.zip.asc	2017-07-06 07:55	819
	apache-ode-war-1.3.7.zip.asc.md5	2017-07-06 07:55	32
	apache-ode-war-1.3.7.zip.asc.sha1	2017-07-06 07:55	40
	apache-ode-war-1.3.7.zip.md5	2017-07-06 07:55	32
	apache-ode-war-1.3.7.zip.sha1	2017-07-06 07:55	40
	apache-ode-war-1.3.8.zip	2018-03-22 05:00	48M
	apache-ode-war-1.3.8.zip.asc	2018-03-22 05:00	833
	apache-ode-war-1.3.8.zip.sha1	2018-03-22 05:00	40
	apache-ode-war-2.0-beta2.zip	2014-10-30 00:31	72M
	apache-ode-war-2.0-beta2.zip.asc	2014-10-30 00:31	197
	apache-ode-war-2.0-beta2.zip.md5	2014-10-30 00:31	61
	apache-ode-war-2.0-beta2.zip.sha1	2014-10-30 00:31	69

Recent download history

- 
apache-ode-war-1.3.7.zip
 12.2/44.5 MB • 21 minutes left
- 
apache-ode-war-1.3.2.zip
 31.8 MB • 12 minutes ago
- 
apache-ode-docs-1.3.2.zip
 Cancelled
- 
UNIT 3.pptx
 3.1 MB • 4 hours ago

Full download history

Activate Windows
 Go to Settings to activate Windows.

Distributed Transactions

A **transaction** is a group of operations performed as a single unit of work.

Example:

Transfer ₹1000

1. Debit Account A
2. Credit Account B

Either both operations succeed or both fail. This is known as an **all-or-nothing operation**.

A **distributed transaction** is a transaction whose operations are executed across multiple computers, databases, servers, or nodes connected through a network.

Example

Customer Places Order----> **Payment Service**-----> **Inventory Service**----->
Shipping Service

All services must complete successfully for the transaction to be considered complete.

Commit Protocols - Two-Phase Commit (2PC)

Most common distributed transaction protocol. The essence of two-phase commit, unsurprisingly, is that it carries out an update in two phases:

1. The prepare phase asks each node if it can promise to carry out the update.
2. The commit phase actually carries it out.

As part of the prepare phase, each node participating in the transaction acquires whatever it needs to assure that it will be able to do the commit in the second phase—for example, any locks that are required.

Once each node is able to ensure it can commit in the second phase, it lets the coordinator know, promising the coordinator that it can and will commit in the second phase.

If any node is unable to make that promise, then the coordinator tells all nodes to roll back, releasing any locks they have, and the transaction is aborted.

Only if all the participants agree to go ahead does the second phase commence—at which point it's expected they will all successfully update.

It is crucial for each participant to ensure the durability of their decisions using pattern like **Write-Ahead Log**. This means that even if a node crashes and subsequently restarts, it should be capable of completing the protocol without any issues.

Phase 1 – Prepare

Coordinator asks participants:

Can you commit?

Participants reply:

YES or NO

Phase 2 – Commit

If all participants reply YES:

COMMIT

Otherwise:

ROLLBACK

Components of Transactions

Client - Initiates the transaction.

Transaction Manager - Coordinates transaction execution.

Responsibilities:

- Begin transaction
- Commit transaction
- Rollback transaction

Resource Managers - Participating databases or services.

**A.C.I.D. properties: Atomicity, Consistency,
Isolation, and Durability**

ACID is an acronym that refers to the set of 4 key properties that define a transaction: **Atomicity**, **Consistency**, **Isolation**, and **Durability**. If a database operation has these ACID properties, it can be called an ACID transaction, and data storage systems that apply these operations are called transactional systems. ACID transactions guarantee that each read, write, or modification of a table has the following properties:

- **Atomicity** - each statement in a transaction (to read, write, update or delete data) is treated as a single unit. Either the entire statement is executed, or none of it is executed. This property prevents data loss and corruption from occurring if, for example, if your streaming data source fails mid-stream.
- **Consistency** - ensures that transactions only make changes to tables in predefined, predictable ways. Transactional consistency ensures that corruption or errors in your data do not create unintended consequences for the integrity of your table.
- **Isolation** - when multiple users are reading and writing from the same table all at once, isolation of their transactions ensures that the concurrent transactions don't interfere with or affect one another. Each request can occur as though they were occurring one by one, even though they're actually occurring simultaneously.
- **Durability** - ensures that changes to your data made by successfully executed transactions will be saved, even in the event of system failure.

OASIS Business Transaction Protocol (BTP)

<https://www.oasis-open.org/committees/business-transaction/documents/primer/Primerhtml/BTP%20Primer%20D1%2020020602.html>

The **Business Transaction Protocol (BTP)** is an XML-based protocol developed by OASIS for coordinating business transactions among multiple distributed and autonomous organizations over the Internet. It is designed for loosely coupled systems where traditional ACID transaction techniques are impractical.

BTP aims to:

- Coordinate transactions across multiple organizations.
- Support loosely coupled systems.
- Manage transaction lifecycle.
- Handle unreliable communication channels.
- Support long-running transactions.
- Provide a foundation for workflow and business process systems.

Main BTP Components

1. Initiator - Starts the transaction.

Example:

- Travel Agent
- Customer
- Bank Application

2. Coordinator - Controls the transaction.

Responsibilities:

- Create transaction context
- Manage participants
- Collect responses
- Decide Confirm or Cancel

3. Participant - A service participating in the transaction.

Examples:

- Flight Booking Service
- Hotel Service
- Car Rental Service
- Payment Service

4. Transaction Context

Contains:

- Transaction ID
- Transaction state
- Participant information

This context travels with service requests.

Two Major Transaction Models

1. Atom - An **Atom** behaves similarly to a traditional transaction.

- All participants see the same outcome.
- All confirm OR all cancel.
- Durable.
- Isolation is relaxed.
- Similar to ACID
- Shorter transactions

Example

Book Flight

Book Hotel

Book Car

If Car Booking Fails

Cancel Flight

Cancel Hotel

Everything succeeds or everything fails.

2. Cohesion - A **Cohesion** is a more flexible transaction.

Characteristics

- Some participants may confirm.
- Some participants may cancel.
- Suitable for long-running business processes.
- Flexible Business Process
- Different results possible

Example

Check Flight Prices

Airline A

Airline B

Airline C

Choose Cheapest Airline

Confirm Airline B

Cancel Airline A

Cancel Airline C

Mixed outcomes are allowed.

BTP and Web Services

- Is XML-based.
- Can work with SOAP.
- Supports Web Services.
- Supports multi-party conversations.
- Supports workflow systems.
- BTP is NOT a workflow language.
- BTP is NOT a business process language.
- BTP is NOT a modeling language.

It only provides reliable transaction coordination.

Advantages of BTP

- Suitable for Internet transactions.
- Supports autonomous organizations.
- No long-term locking.
- Handles unreliable networks.
- Supports long-running business activities.
- Supports distributed web services.

A **Compound Transaction** is a long-running transaction that consists of multiple smaller sub-transactions executed across different systems, services, or databases. Unlike traditional ACID transactions, a compound transaction may span a long period and uses **compensation mechanisms** instead of immediate rollback to handle failures.

A Compound Transaction is a collection of related transactions that work together to complete a business process. If one step fails, previously completed steps are undone using compensation actions.

Example of a Compound Transaction

A **travel booking system** is a common example of a compound transaction. Suppose a customer wants to book a complete travel package that includes a flight ticket, hotel reservation, and taxi service. The process starts by booking the flight, then reserving the hotel, and finally arranging the taxi. Each of these tasks is handled by different service providers and can be considered an independent sub-transaction. If all three bookings are successful, the entire transaction is completed successfully. However, if the flight and hotel are booked successfully but the taxi booking fails, the system cannot simply roll back the already completed bookings because they may have been confirmed by external organizations. Instead, the system performs **compensation actions**, such as canceling the flight reservation and hotel booking, to restore the system to a consistent business state. Thus, a compound transaction consists of multiple related transactions executed over a period of time, where failures are handled through compensation rather than traditional rollback.

Important BTP Terms

Enroll - Participant agrees to participate in a transaction.

Example: Hotel Service joins transaction

Prepare - Participant promises it can complete the transaction.

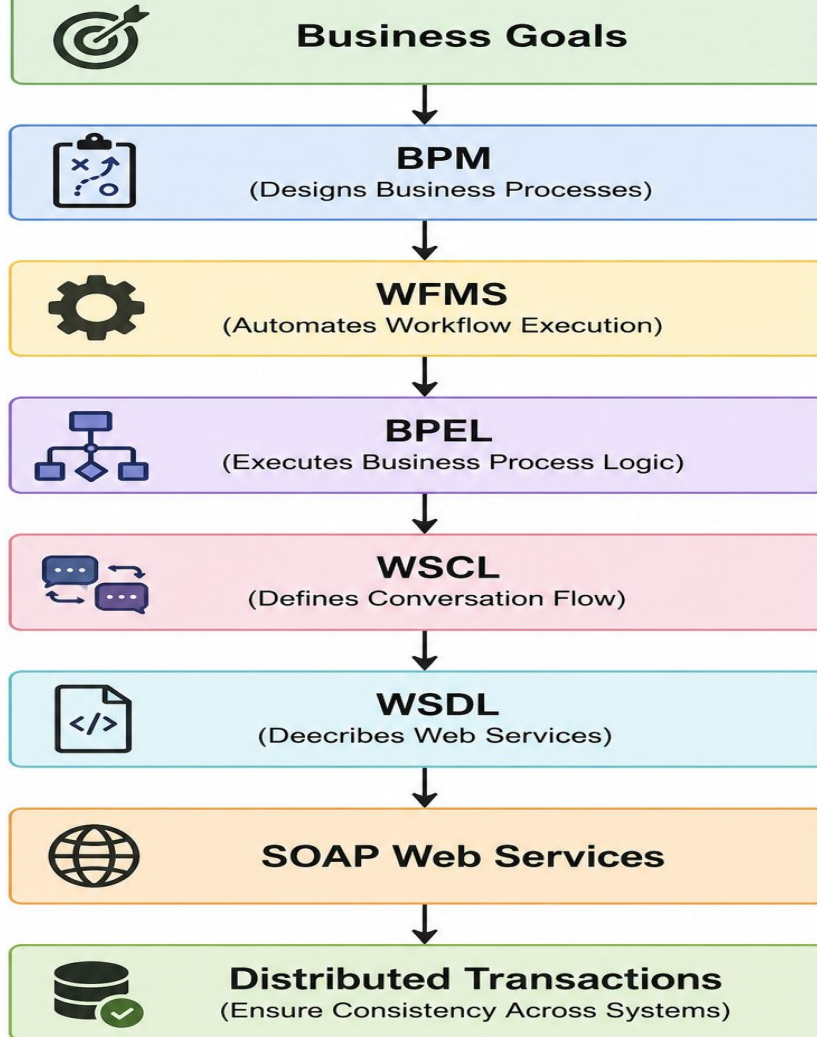
Example: Room reserved for 24 hours

Confirm - Transaction is successfully completed.

Cancel - Transaction is rolled back or compensated.

Relation with BPEL and OASIS BTP

- **BPEL** uses compensation handlers to manage compound transactions.
- **OASIS Business Transaction Protocol (BTP)** was designed specifically to support compound transactions in distributed business environments.



An Example of WSCL Testing >Java Project

StudentResultService.java

```
package com.student;
import javax.jws.WebMethod;
import javax.jws.WebService;
@WebService
public class StudentResultService {
    private boolean loggedIn = false;
    @WebMethod
    public String login(String username, String password) {
        if(username.equals("admin") &&
            password.equals("123")) {
            loggedIn = true;
            return "Login Successful";
        }
    }
}
```

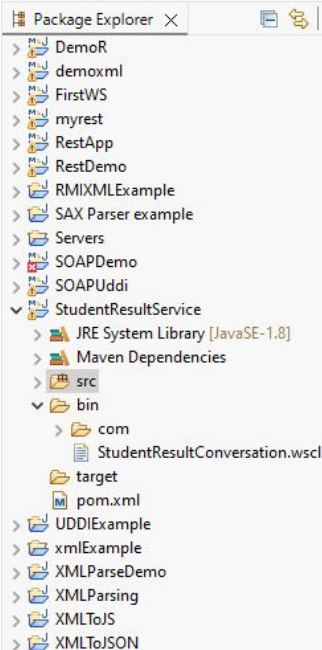
```
return "Invalid Username or Password";
}
@WebMethod
public String getResult(String enrollmentNo) {
    if(!loggedIn) {
        return "Please Login First";
    }
    if(enrollmentNo.equals("101")) {
        return "Enrollment No : 101\n"
            + "Name : John\n"
            + "Marks : 85\n"
            + "Result : PASS";
    }
    return "Student Record Not Found";
}
@WebMethod
public String logout() {
    loggedIn = false;
    return "Logout Successful";
}
}
```

StudentResultPublisher.java

```
package com.student;
import javax.xml.ws.Endpoint;
public class StudentResultPublisher {
    public static void main(String[] args) {
        Endpoint.publish(
            "http://localhost:8080/StudentResultService",
            new StudentResultService());
        System.out.println(
            "SOAP Student Result Service Started...");
    }
}
```

Src > new > file > StudentResultConversation.wscl

```
<?xml version="1.0" encoding="UTF-8"?>
<Conversation name="StudentResultConversation">
  <Interaction
    name="Login"
    type="ReceiveSend"/>
  <Interaction
    name="GetResult"
    type="ReceiveSend"/>
  <Interaction
    name="Logout"
    type="ReceiveSend"/>
  <Transition
    fromInteraction="Login"
    toInteraction="GetResult"/>
  <Transition
    fromInteraction="GetResult"
    toInteraction="Logout"/>
</Conversation>
```



StudentResultService.java StudentResultPublisher.java StudentResultConversation.wscl StudentResultService/pom.xml

```
<!-- JAX-WS API -->
<dependency>
  <groupId>javax.xml.ws</groupId>
  <artifactId>jaxws-api</artifactId>
  <version>2.3.1</version>
</dependency>

<!-- JAX-WS Runtime -->
<dependency>
  <groupId>com.sun.xml.ws</groupId>
  <artifactId>jaxws-rt</artifactId>
  <version>2.3.3</version>
</dependency>

<!-- JAXB -->
<dependency>
  <groupId>javax.xml.bind</groupId>
  <artifactId>jaxb-api</artifactId>
  <version>2.3.1</version>
```

Overview Dependencies Dependency Hierarchy Effective POM pom.xml

Problems Declaration Console

StudentResultPublisher [Java Application] C:\Users\monikatiwari.INDUS\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.10.v20260205-0638\jre\bin\javaw.exe (Jun 20, 2026 1:31:16 PM com.sun.xml.ws.server.MonitorBase createRoot
INFO: Metro monitoring rootname successfully set to: com.sun.metro:pp=/,type=wsEndpoint,name=StudentResultServiceService-StudentResultServicePo
SOAP Student Result Service Started...

Activate Windows
Go to Settings to activate Windows.

Writable

Insert

27: 22: 1010



35°C Sunny

ENG
US1:34 PM
6/20/2026

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<!-- Published by JAX-WS RI (https://github.com/eclipse-ee4j/metro-jax-ws). RI's version is JAX-WS RI 2.3.3 git-revision#b4c5bb6. -->
<!-- Generated by JAX-WS RI (https://github.com/eclipse-ee4j/metro-jax-ws). RI's version is JAX-WS RI 2.3.3 git-revision#b4c5bb6. -->
<?xml version='1.0' encoding='UTF-8'?>
<definitions xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd" xmlns:wsp="http://www.w3.org/ns/ws-policy"
xmlns:wsp1_2="http://schemas.xmlsoap.org/ws/2004/09/policy" xmlns:wsam="http://www.w3.org/2007/05/addressing/metadata" xmlns:soap="http://schemas.xmlsoap.org/soap/"
xmlns:tns="http://student.com/" xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://schemas.xmlsoap.org/wsdl/" targetNamespace="http://student.com/"
name="StudentResultServiceService">
  <types>
    <xsd:schema>
      <xsd:import namespace="http://student.com/" schemaLocation="http://localhost:8080/StudentResultService?xsd=1"/>
    </xsd:schema>
  </types>
  <message name="logout">
    <part name="parameters" element="tns:logout"/>
  </message>
  <message name="logoutResponse">
    <part name="parameters" element="tns:logoutResponse"/>
  </message>
  <message name="login">
    <part name="parameters" element="tns:login"/>
  </message>
  <message name="loginResponse">
    <part name="parameters" element="tns:loginResponse"/>
  </message>
  <message name="getResult">
    <part name="parameters" element="tns:getResult"/>
  </message>
  <message name="getResultResponse">
    <part name="parameters" element="tns:getResultResponse"/>
  </message>
  <portType name="StudentResultService">
    <operation name="logout">
      <input wsam:Action="http://student.com/StudentResultService/logoutRequest" message="tns:logout"/>
      <output wsam:Action="http://student.com/StudentResultService/logoutResponse" message="tns:logoutResponse"/>
    </operation>
    <operation name="login">
      <input wsam:Action="http://student.com/StudentResultService/loginRequest" message="tns:login"/>
      <output wsam:Action="http://student.com/StudentResultService/loginResponse" message="tns:loginResponse"/>
    </operation>
    <operation name="getResult">
      <input wsam:Action="http://student.com/StudentResultService/getResultRequest" message="tns:getResult"/>
      <output wsam:Action="http://student.com/StudentResultService/getResultResponse" message="tns:getResultResponse"/>
    </operation>
  </portType>
</definitions>
```

Activate Windows
Go to Settings to activate Windows.

SoapUI 5.9.1

File Project Suite Case Step Tools Desktop Help

Empty SOAP REST Import Save All Forum Trial Preferences Proxy Endpoint Explorer Search Forum Online Help

Navigator

- my
 - REST Project 4
 - REST Project 5
 - UDDIExample
 - WSCLDemo
 - StudentResultServicePortBinding
 - getResult
 - login
 - Request 1**
 - logout

Request 1

Request 1

http://localhost:8080/StudentResultService

Raw XML

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Header/>
  <soapenv:Body>
    <stud:login>
      <!--Optional:-->
      <arg0>admin</arg0>
      <!--Optional:-->
      <arg1>123</arg1>
    </stud:login>
  </soapenv:Body>
</soapenv:Envelope>
```

Raw XML

```
<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body>
    <ns2:loginResponse xmlns:ns2="http://student.com/">
      <return>Login Successful</return>
    </ns2:loginResponse>
  </S:Body>
</S:Envelope>
```

Headers (4) Attachments (0) SSL Info WSS (0) JMS (0)

response time: 43ms (236 bytes)

Request Properties

Property	Value
Name	Request 1
Description	

Properties

SoapUI log http log jetty log error log wsrm log memory log

Activate Windows
Go to Settings to activate Windows.

SoapUI 5.9.1

FileProjectSuiteCaseStepToolsDesktopHelp

EmptySOAPRESTImportSaveAllForumTrialPreferencesProxy

Endpoint Explorer

Search Forum

Online Help

my

REST Project 4

REST Project 5

UDDIExample

WSCLDemo

StudentResultServicePortBinding

getResult

Request 1

login

Request 1

logout

Request 1

Request 1

http://localhost:8080/StudentResultService

Raw XML

```
<?xml version='1.0' encoding='UTF-8'>
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Header/>
  <soapenv:Body>
    <stud:getResult>
      <!--Optional-->
      <arg0>101</arg0>
    </stud:getResult>
  </soapenv:Body>
</soapenv:Envelope>
```

Raw XML

```
<?xml version='1.0' encoding='UTF-8'>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/">
  <S:Body>
    <ns2:getResultResponse xmlns:ns2="http://student.com/">
      <return>Enrollment No : 101
      Name : John
      Marks : 85
      Result : PASS</return>
    </ns2:getResultResponse>
  </S:Body>
</S:Envelope>
```

Headers (4)Attachments (0)SSL InfoWSS (0)JMS (0)

resp: response time: 7ms (284 bytes)

1: 1

Request Properties

Property	Value
Name	Request 1
Description	

Properties

SoapUI loghttp logjetty logerror logwsrm logmemory log

